

Exercise 1 — Employee Database (JDBC + MySQL)

Database schema

- Database: `EmployeeDB`.
- Table `Employees`: `id` (INT, AUTO_INCREMENT, PRIMARY KEY), `name` (VARCHAR(50)), `age` (INT), `department` (VARCHAR(50)), `salary` (DOUBLE).

Connecting

- Connect once at startup with `DriverManager.getConnection("jdbc:mysql://localhost:3306/EmployeeDB", "root", "")` (adjust the username/password to your local MySQL setup), keep the connection open for the whole session, and close it when the program exits (a try-with-resources on the connection, or a finally block, both work).

Menu

- Loop showing: 1) Add employee, 2) List all employees, 3) Update employee by id, 4) Delete employee by id, 5) Search by name or department, 6) Exit. Keep looping until Exit is chosen.

Function behaviour

- Add: prompt for name, age, department, and salary from the keyboard, then insert with a `PreparedStatement` that binds all four values as parameters (not string concatenation) — this avoids SQL injection and correctly handles names containing quotes or special characters.
- List: `SELECT * FROM Employees`, print one row per line in a readable format.
- Update by id: prompt for the id, then the new field values, and run a parameterized `UPDATE ... WHERE id = ?`; print the number of affected rows — 0 means that id didn't exist.
- Delete by id: prompt for the id, run a parameterized `DELETE FROM Employees WHERE id = ?`, and report success or failure based on the affected-row count.
- Search by name or department: prompt for a single search term, then run one query that checks both columns with wildcards on both sides, e.g. `WHERE name LIKE ? OR department LIKE ?` with the parameter bound as `"%" + term + "%"` for each placeholder — so the same search box matches either field. Print all matches, or a "No employees found" message if there are none.

Exercise 2 — Product Database (JDBC + MySQL)

Database schema

- Database: `ProductDB`.
- Table `Products`: `id` (INT, AUTO_INCREMENT, PRIMARY KEY), `name` (VARCHAR(100)), `category` (VARCHAR(50)), `price` (DOUBLE), `quantity` (INT).

Menu and behaviour

- Same structure as a typical CRUD console app: 1) Add product, 2) List all products, 3) Update product by id, 4) Delete product by id, 5) Search by name or category, 6) Exit.
- Use `PreparedStatement` with bound parameters for every statement that includes user-supplied values (add, update, delete, search) — never build SQL by string concatenation.
- Search by name or category: prompt for a single search term, then run one query that checks both columns with wildcards on both sides, e.g. `WHERE name LIKE ? OR category LIKE ?`, with the parameter bound as `"%" + term + "%"` for each placeholder.

Hashtable requirement

- When listing all products (option 2), don't print directly from the `ResultSet`. Instead, load every row into a `Hashtable<Integer, Product>` first, keyed by `id`, and only then iterate the Hashtable's values to print the list. This is a deliberate requirement of the exercise (practicing Hashtable), even though a plain List would normally be the simpler choice here.

DAO pattern (required)

- `Product`: a plain model class with the five fields, getters/setters, and `toString()`.
- `ProductRepository`: an interface declaring the CRUD method signatures — `add`, `findAll`, `findById`, `update`, `delete`, `search`.
- `ProductRepositoryImpl`: implements `ProductRepository` using JDBC — this is the only class that actually talks to the database.
- The menu/console code should depend only on the `ProductRepository` interface type, never directly on JDBC calls or on `ProductRepositoryImpl` — that separation is the actual point of the DAO pattern.

Exercise 3 — Employee Management Application (Design Pattern & Internationalization)

Database schema

- Database: `EmployeeDB`.
- Table `tblNhanVien`: `id` (INT, PRIMARY KEY), `name` (NVARCHAR(64)), `yearOfBirth` (INT), `address` (VARCHAR(256)), `phone` (VARCHAR(11)), `salary` (INT).

Menu

- Loop showing: 1) Add new employee, 2) List employees, 3) Search employee, 4) Change language (Vietnamese ↔ English), 5) Exit.

Connection class (Singleton)

- Create a class, e.g. `DBConnection`, with a private static field holding the one instance, a private constructor that opens the JDBC `Connection`, and a public static `getInstance()` method that creates the instance the first time it's called and returns the same one on every later call — so the whole application shares exactly one Connection object.

DAO pattern

- An `EmployeeDAO` interface declaring the CRUD/search method signatures, and an `EmployeeDAOImpl` class implementing them with JDBC, using `PreparedStatement` for every query that takes a parameter. The menu code should call only the interface, never the JDBC implementation directly.

Internationalization (Locale + ResourceBundle)

- Create two property files with the same set of keys, e.g. `messages_en.properties` and `messages_vi.properties` — one key per piece of UI text (menu items, prompts, success/error messages), e.g. `menu.add=Add new employee` in the English file, with the equivalent Vietnamese phrasing under the same key in the Vietnamese file.
- Load the active bundle with `ResourceBundle.getBundle("messages", locale)`, where `locale` is `Locale.ENGLISH` or a Vietnamese locale such as `new Locale("vi", "VN")`.
- Store the current Locale in a field on the main class. Option 4 toggles it and reloads the bundle. Every menu line and prompt should be read from the bundle rather than hard-coded, so the whole UI switches language

immediately when option 4 is chosen. The selection only needs to last for the current run — it doesn't need to be saved between runs.

Exercise 4 — Advanced JDBC: Goods (Inventory) Management

Database schema

- Table **Products**: **productCode** (VARCHAR, PRIMARY KEY), **productName** (VARCHAR), **manufacturer** (VARCHAR), **quantity** (INT), **originalPrice** (DECIMAL, in VND), **vatRate** (DECIMAL, stored as a percentage, e.g. **10.00** for 10%).

Menu

- Loop showing: 1) Stock in, 2) Stock out, 3) Inventory, 4) Update, 5) Exit.

Stock in — PreparedStatement + batch insert

- Prompt for how many line items are being received in this batch, then for each: product code, name, manufacturer, quantity, original price, VAT rate.
- Add each item as a parameterized **INSERT ... ON DUPLICATE KEY UPDATE quantity = quantity + VALUES(quantity)** call to a single **PreparedStatement** via **addBatch()** (the ON-DUPLICATE-KEY clause means re-stocking an existing product code adds to its quantity instead of erroring on the duplicate primary key), and execute the whole batch in one **executeBatch()** call once all items are entered — this is the batch-insert requirement.

Stock out — stored procedure

- Create a MySQL stored procedure, e.g. **sp_stock_out(IN p_code VARCHAR(50), IN p_qty INT)**, that checks the current quantity for **p_code**; if it's less than **p_qty**, it raises an error with **SIGNAL SQLSTATE '45000'** and a descriptive message (which cancels the transaction); otherwise it decrements the quantity by **p_qty**.
- Call it from Java with a **CallableStatement** (e.g. **{call sp_stock_out(?, ?)}**); catch the **SQLException** raised by the signal and print a warning to the console instead of letting a stack trace through.

Inventory and Update

- Inventory: list every product with all columns. No user input is involved in this query, so a plain **Statement** or an unparameterized **PreparedStatement** both work.
- Update: prompt for a product code, then new values for the editable fields — name, manufacturer, original price, VAT rate (not quantity, since quantity only ever changes via stock in/out) — and update the row with a parameterized **PreparedStatement**.

Exercise 5 — Student Management Console Application (JDBC)

Build a student-management console application against a database named **qlsv**, with a table **tblSinhVien**.

Database schema (tblSinhVien)

- id** — INT(11), AUTO_INCREMENT, PRIMARY KEY, NOT NULL.
- rollnumber** — VARCHAR(8), NOT NULL.
- name** — VARCHAR(256), NOT NULL.
- address** — VARCHAR(512), NULL allowed.

- `phone` — VARCHAR(11), NULL allowed.
- `gender` — BIT(1), NOT NULL.

Part 1 — Connect to the Database and Report Connection Status

Goal: create a Java console application that connects to a database named `qlsv` and prints whether the connection succeeded.

Add the MySQL JDBC driver to your project

- Download the MySQL Connector/J jar (e.g. `mysql-connector-java-x.x.x-bin.jar`) and add it to your project's build path. In NetBeans: right-click the project → Properties → Libraries → Add JAR/Folder → select the driver jar. (For SQL Server instead of MySQL, add `sqljdbc4.jar` the same way.)
- Keep the jar inside your project (e.g. in a `libs` folder) rather than referencing it from an arbitrary location on disk, so the reference doesn't break if the project moves.

MainClass.java — a minimal connection test:

```
package demo.jp2.lab06;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;

public class MainClass {

    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.jdbc.Driver");
            Connection conn = null;
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/qlsv",
                "root", "");
            conn.close();
            System.out.println("Connected to the database successfully");
        } catch (ClassNotFoundException | SQLException ex) {
            Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}
```

Connection details

- A locally installed MySQL server normally listens on `localhost`, port 3306, with username `root` and an empty password by default — adjust the connection string if your setup differs.
- A locally installed SQL Server instance is normally addressed as `<machine-name>\<instance-name>` and is used the same way once you're connected.

Running it

- Before the `qlsv` database exists, running this will throw a `MySQLSyntaxErrorException: Unknown database 'qlsv'` — that's expected.
- Create the `qlsv` database (and the `tblSinhVien` table, using the schema above), then run the program again: it should print "Connected to the database successfully" and the build should succeed.

Part 2 — Insert Student Records from Keyboard Input

Goal: add a function that reads one student's information from the keyboard and inserts it into `tblSinhVien` using a `Statement`.

SinhVien.java — the student model, plus the keyboard-input and insert logic:

```
package demo.jp2.lab06;
```

```

import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class SinhVien {

    int id;
    String rollNumber;
    String name;
    String address;
    String phoneNumber;
    boolean gender;

    private void enterInfo() {
        System.out.println("Enter student information:");
        Scanner input = new Scanner(System.in);
        System.out.println("Student ID: ");
        this.rollNumber = input.nextLine();
        System.out.println("Full name: ");
        this.name = input.nextLine();
        System.out.println("Address: ");
        this.address = input.nextLine();
        System.out.println("Phone number: ");
        this.phoneNumber = input.nextLine();
        System.out.println("Gender (Male = 1 | Female = 0): ");
        int gend = input.nextInt();
        if (gend == 1) {
            this.gender = true;
        } else {
            this.gender = false;
        }
    }

    public void insert(Statement st) throws SQLException {
        enterInfo();
        String sql = "INSERT INTO `tblsinhvien`("
            + "`rollnumber`, `name`, `address`, `phone`, `gender`)"
            + "VALUES ("
            + "'" + this.rollNumber + "',"
            + "'" + this.name + "',"
            + "'" + this.address + "',"
            + "'" + this.phoneNumber + "',"
            + "'" + this.gender + "'"
            + ")";
        int row = st.executeUpdate(sql);
        if (row > 0) {
            System.out.println("Added successfully");
        } else {
            System.out.println("Failed, please check your data");
        }
    }
}

```

MainClass.java — the menu, and the wiring for option 1. Note that **sv** is created once, before the loop, so the same object is available to every case in the switch statement across every pass through the menu — including the update/delete cases added in Part 4 below:

```

package demo.jp2.lab06;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
import java.util.logging.Level;
import java.util.logging.Logger;

public class MainClass {

```

```

Connection conn = null;

public void connect() {
    try {
        Class.forName("com.mysql.jdbc.Driver");
        conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/qlsv",
            "root", "");
        System.out.println("Connected to the database successfully");
    } catch (ClassNotFoundException | SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private int showMenu() {
    System.out.println("==== MENU =====");
    System.out.println("1. Enter student information");
    System.out.println("2. List of students");
    System.out.println("3. Edit student");
    System.out.println("4. Delete student");
    System.out.println("5. Exit");
    System.out.println("Please choose from 1 -> 5");
    Scanner input = new Scanner(System.in);
    return input.nextInt();
}

public static void main(String[] args) {
    MainClass main = new MainClass();
    main.connect();
    SinhVien sv = new SinhVien();
    int choose;
    do {
        choose = main.showMenu();
        Statement st = null;
        try {
            st = main.conn.createStatement();
        } catch (SQLException ex) {
            Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
        }
        switch (choose) {
            case 1:
                try {
                    sv.insert(st);
                } catch (SQLException ex) {
                    Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
                }
                break;
            case 2:
                break;
            case 3:
                break;
            case 4:
                break;
        }
    } while (choose > 0 && choose < 5);
    try {
        main.conn.close();
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
}
}

```

Expected console flow

- Choosing option 1 prompts: "Enter student information:", then "Student ID:", "Full name:", "Address:", "Phone number:", "Gender (Male = 1 | Female = 0):" in order.

- Example input: **JF150801 / Jang Hang Ra / Seoul / 1234567890 / 0** (Female).
- After the last field, the program prints "Added successfully" (assuming the insert affected a row) and redisplay the menu.

Part 3 — Load and Display the List of Students

Goal: add a function that loads every student from the database and prints the list.

Add this method to SinhVien.java (and add an import for ResultSet):

```
// add to SinhVien.java (also add "import java.sql.ResultSet;" to its imports)

public void load(Statement st) throws SQLException {
    String sql = "SELECT * FROM `tblsinhvien`";
    ResultSet result = st.executeQuery(sql);
    System.out.println("-----");
    while (result.next()) {
        String id = result.getString("id");
        String rollnumber = result.getString("rollnumber");
        String name = result.getString("name");
        String address = result.getString("address");
        String phone = result.getString("phone");
        String gender = "Male";
        if (result.getInt("gender") == 0) {
            gender = "Female";
        }
        System.out.println(id + "|" + rollnumber + "|" + name
            + " - " + address + " - " + phone + " - " + gender);
    }
    System.out.println("-----");
}
```

Wire it into MainClass's switch statement:

```
// in MainClass.java's switch statement

case 2:
    try {
        sv.load(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
    break;
```

Expected output

- Format: **id| rollnumber| name - address - phone - gender**, one line per student, between two rules of dashes. Example, continuing from the record inserted in Part 2:

```
1| C1505001| Ma Sieu - Tay Luong - 19001265 - Male
2| C1505002| Dieu Thuyen - To Chau - 19001001 - Female
5| C1505003| Ma Sieu - Tay Luong - 19001265 - Male
6| JF150801| Jang Hang Ra - Seoul - 1234567890 - Female
```

Part 4 — Edit and Delete Student Records

Goal: add functions to edit a student's information and to delete a student, both looked up by rollnumber.

Add these three methods to SinhVien.java:

```
// add to SinhVien.java

private void editInfo() {
    System.out.println("Enter updated information:");
    Scanner input = new Scanner(System.in);
    System.out.println("Full name: ");
    this.name = input.nextLine();
}
```

```

        System.out.println("Address: ");
        this.address = input.nextLine();
        System.out.println("Phone number: ");
        this.phoneNumber = input.nextLine();
        System.out.println("Gender (Male = 1 | Female = 0): ");
        int gend = input.nextInt();
        if (gend == 1) {
            this.gender = true;
        } else {
            this.gender = false;
        }
    }

    public void update(Statement st) throws SQLException {
        System.out.println("Update student information");
        System.out.println("Enter the student ID to edit: ");
        Scanner input = new Scanner(System.in);
        String rollNumber = input.nextLine();
        editInfo();
        String sql = "UPDATE `tblsinhvien` SET "
            + "`name` = '" + this.name + "',"
            + "`address` = '" + this.address + "',"
            + "`phone` = '" + this.phoneNumber + "',"
            + "`gender` = '" + this.gender + "' WHERE "
            + "`rollnumber` = '" + rollNumber + "'";
        int row = st.executeUpdate(sql);
        if (row > 0) {
            System.out.println("Updated successfully");
        } else {
            System.out.println("Failed, please check your data");
        }
    }

    public void delete(Statement st) throws SQLException {
        System.out.println("Enter the student ID to delete: ");
        Scanner input = new Scanner(System.in);
        String rollNumber = input.nextLine();
        String sql = "DELETE FROM `tblsinhvien` WHERE "
            + "`rollnumber` = '" + rollNumber + "'";
        int row = st.executeUpdate(sql);
        if (row > 0) {
            System.out.println("Deleted successfully");
        } else {
            System.out.println("Failed, please check your data");
        }
    }
}

```

Wire both into MainClass's switch statement:

```

// in MainClass.java's switch statement

case 3:
    try {
        sv.update(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
    break;
case 4:
    try {
        sv.delete(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
    break;

```

Expected output

- Option 3, editing **JF150801** with new values **Giao Hop Phat 1 / Motel / 69696969 / 1** (Male): prints "Update student information", then "Enter the student ID to edit:", then the same four update prompts as editInfo(), then "Updated successfully".
- Option 4, deleting **C1505003**: prints "Enter the student ID to delete:", then "Deleted successfully".

A known limitation in this version

- All four SQL statements above build the query text by concatenating strings together, which is vulnerable to SQL injection and breaks if a name or address contains a single quote. As a follow-up improvement, rewrite insert(), update(), and delete() to use PreparedStatement with bound (?) parameters instead of concatenation — the same approach used elsewhere in this set.
-